

## Actividad\_7\_Cierre Final\_Transferencia de Estilo Neuronal



**Autores:** Ruben Perez, Marcos Del Saz

**Asignatura:** Sistemas de Aprendizaje Automático

**Fecha:**27/05/2026

**BLOQUE 1:** Inferencia Estilo Feed-Forward (Fast NST)

## Bloque 1: Fast NST (Caja Negra)



**Respuesta a la Pregunta 1.1:** La diferencia principal está en cómo trabajan "bajo el capó". El modelo *Feed-Forward* de TensorFlow Hub es súper rápido porque ya está entrenado para hacer la mezcla de golpe; le pasas las dos fotos y te escupe el resultado en una sola pasada, en cuestión de milisegundos. Por otro lado, el NST tradicional no "adivina" la imagen final directamente. Lo que hace es coger un lienzo en blanco (o con ruido inicial) y usar una red como la VGG19 a modo de "juez" para medir cuánto se equivoca respecto a lo que queremos lograr. Luego, a través de un bucle iterativo, va corrigiendo y pintando los píxeles del lienzo poco a poco hasta que el resultado final encaja con el estilo y el contenido.

**Respuesta a la Pregunta 1.2:** Esto ocurre porque las redes usan "filtros" de un tamaño fijo (por ejemplo, pequeñas ventanas de 3x3 píxeles) para analizar la imagen. Si la foto de entrada es de baja resolución (como 256x256), ese cuadradito de 3x3 abarca una parte bastante grande de nuestra cara, como un ojo o la nariz. Por eso, el algoritmo aplica unas pinceladas muy gruesas y vistosas. En cambio, si le pasamos una imagen enorme (1024x1024), ese mismo filtro de 3x3 ahora solo cubre un detalle minúsculo, como un poro de la piel. Al intentar aplicar el estilo a esa escala, las pinceladas se vuelven súper finas y densas, haciendo que el resultado final pierda la forma de trazo artístico y parezca más bien una textura con mucho ruido.

## BLOQUE 2: Optimización de Lienzo y Extracción Jerárquica con VGG19

Bloque 2 | Paso 100 | Loss: 1447.41



```
@tf.function
def paso_entrenamiento(imagen):
    with tf.GradientTape() as tape:
        salidas_estilo, salidas_contenido = extraer_caracteristicas(imagen)

        loss_cont = tf.add_n([tf.reduce_mean((salidas_contenido[i] - targets_contenido[i])**2) for i in range(len(salidas_contenido))])
        loss_est = tf.add_n([tf.reduce_mean((salidas_estilo[i] - targets_estilo[i])**2) for i in range(len(salidas_estilo))])

        loss_total = (peso_contenido * loss_cont) + (peso_estilo * loss_est)

    gradientes = tape.gradient(loss_total, imagen)
    optimizador.apply_gradients([(gradientes, imagen)])
    imagen.assign(tf.clip_by_value(imagen, clip_value_min=0.0, clip_value_max=1.0))
    return loss_total
```

**Respuesta a la Pregunta 2.1:** En la línea `gradientes = tape.gradient(loss_total, imagen)`, lo que hace el `GradientTape` es calcular los gradientes basándose en los **píxeles de nuestro lienzo**, no en los pesos de la red VGG19. Básicamente, le damos la vuelta a lo que hacemos normalmente. En un entrenamiento clásico para clasificar imágenes, la foto de entrada es fija y lo que ajustamos son los "conocimientos" (los pesos) de la red para que aprenda. Aquí hacemos lo contrario: congelamos la red para que actúe únicamente como "crítico de arte" (nuestra función de pérdida) y lo que vamos retocando y actualizando paso a paso son los píxeles del lienzo, hasta que la imagen encaje con lo que buscamos.

**Respuesta a la Pregunta 2.2:** Al pasar la capa de contenido de `block4_conv2` a una mucho más profunda como `block5_conv2`, nuestras caras se deforman un montón y pierden su

forma natural. Esto pasa porque las primeras capas de la red se fijan en detalles exactos como líneas, bordes y dónde está cada cosa. Sin embargo, las capas más profundas se quedan con "la idea" general (saben que hay un rostro, pero no les importa exactamente dónde están los ojos o la boca). Al usar el bloque 5, le damos demasiada libertad a la red; permite que nuestras caras se distorsionen mientras siga reconociendo el concepto abstracto de que ahí hay una persona.

### **Respuesta a la Pregunta 2.3:**

- **Caso A ( $\alpha \gg \beta$ ):** Aquí el contenido manda. El resultado es prácticamente nuestra foto original de la webcam. El estilo del cuadro es tan débil que no llega a pintar texturas ni cambiar los colores de forma notable.
- **Caso B ( $\beta \gg \alpha$ ):** Aquí el estilo satura y se come al contenido. Nuestras caras desaparecen por completo porque se pierde toda la estructura. El lienzo termina siendo una mancha abstracta con los colores, trazos y patrones de *El Grito* de Munch, pero sin que se reconozca absolutamente ninguna forma.

**Respuesta a la Pregunta 2.4:** Si miramos cómo evoluciona la imagen paso a paso, lo primero que se dibuja y asienta en el lienzo es la estructura base: nuestras siluetas y la forma macroscópica de las caras (el contenido). Después, conforme avanza el bucle, es cuando empiezan a emerger los colores, los trazos y las texturas del cuadro (el estilo). Esto tiene sentido porque el cálculo del error de contenido es muy estricto desde el principio y "castiga" rápido cualquier cambio que rompa nuestra geometría. En cambio, inyectar el estilo (que se calcula con la Matriz de Gram) es un proceso más complejo que necesita más iteraciones para ir tejiendo esos patrones artísticos poco a poco sin destrozarse las caras que ya se habían formado.

## **BLOQUE 3: Ingeniería de Prompts y Co-Creación con IA**

Bloque 3 (Refinado + Suavizado) | Paso 100 | Loss: 3081863692288.00



### Respuesta a la Pregunta 3.1 (Bitácora de Prompts):

Los *prompts* que realmente marcaron la diferencia para desatascar el problema fueron estos dos:

1. Primero atacamos el problema de raíz preguntando por los datos: *"Auditoría de Datos: Actualmente mis píxeles están en el rango dinámico [0.0, 1.0]. Explicame a nivel matemático cómo afecta este rango frente al original [0.0, 255.0] y cómo influye la ausencia del preprocesamiento nativo..."* y aprovechamos para pedirle que nos sustituyera las capas MaxPooling2D por AveragePooling2D.
2. Después, le dimos un objetivo visual muy claro: *"Quiero que la imagen sea lo más parecida a la imagen final que nos ha dado el bloque 1, una imagen mas suave y con menos ruido... que se asemeje lo maximo posible al estilo del cuadro del grito".*

A nivel matemático, la explicación que nos dio la IA tiene todo el sentido. Resulta que la red VGG19 se entrenó originalmente usando imágenes en un rango de 0 a 255 y con los valores de color ajustados (restando la media en formato BGR). Al meterle nosotros los píxeles "a pelo" en rango [0.0, 1.0] sin aplicarle su preprocess\_input nativo, la red recibía señales

súper débiles. Esto provocaba que los gradientes fueran minúsculos, lo que explica por qué nuestro lienzo simplemente se emborronaba y no avanzaba.

Además, nos aclaró por qué debíamos cambiar el *pooling*. El MaxPooling es muy agresivo porque descarta mucha información (solo se queda con el píxel más alto de cada zona), lo que "rompe" el camino por donde fluye el aprendizaje (el gradiente). Al cambiarlo por AveragePooling, hacemos que el gradiente viaje de una forma mucho más suave, continua y repartida. Este pequeño cambio en la arquitectura fue lo que nos permitió conseguir unas texturas infinitamente mejores y cargarnos todo ese ruido visual.

"Hola. Estoy trabajando en un proyecto de Transferencia de Estilo Neuronal (NST) usando TensorFlow y Keras. He implementado un bucle de optimización explícito usando la red VGG19 preentrenada, pero tengo un problema: el lienzo simplemente se emborrona y desenfoca de forma tímida, estancándose en un mínimo local.

Necesito auditar y depurar mi código contigo basándonos en dos puntos críticos:

1. Auditoría de Datos: Actualmente mis píxeles están en el rango dinámico  $[0.0, 1.0]$ . Explícame a nivel matemático cómo afecta este rango frente al original  $[0.0, 255.0]$  y cómo influye la ausencia del preprocesamiento nativo de ImageNet (`preprocess_input`) en la fuerza de los gradientes que genera la VGG19. ¿Cómo lo corrijo en el código?

2. Optimización Arquitectónica: Entiendo que la VGG19 original usa capas MaxPooling2D que destruyen el flujo de gradientes para las texturas finas. ¿Cómo puedo reconstruir el modelo de Keras de forma funcional para sustituirlas por AveragePooling2D sin romper el registro interno de nombres de las capas (para poder seguir usando mis salidas `blockX_convX`)?

Por favor, dame la explicación técnica y el código paso a paso para aplicar estas dos correcciones a mi bucle de extracción y optimización."

quiero que la imagen sea lo más parecida a la imagen final que nos ha dado el bloque 1, una imagen mas suave y con menos ruido que la que nos has dado en el bloque 3, que se asemeje lo maximo posible al estilo del cuadro del grito

### Respuesta a la Pregunta 3.2 (Depuración de Errores):

Nada más intentar arrancar, nos saltó un error de ejecución al cargar la imagen del cuadro. El fallo venía dado por incompatibilidades con el formato y las dimensiones del archivo. Para salir del paso, le pedimos a la IA que modificara el código de preprocesamiento para hacerlo bastante más "friendly" y flexible, de modo que no fuera tan exigente con las imágenes de entrada.

La solución técnica que nos propuso para arreglar esto fue implementar la función `tf.image.decode_image`. Esta función es súper útil porque detecta y soporta diferentes formatos de imagen (JPEG, PNG, etc.) sobre la marcha. Además, lo combinó con `tf.image.resize` para forzar que cualquier imagen que le pasemos se ajuste automáticamente al tamaño exacto que necesita la red. De esta forma, nos aseguramos de que las dimensiones siempre cuadren y evitamos que el tensor dé problemas y rompa la ejecución del programa en el primer paso.

me da un error a la hora de cargar la imagen del cuadro por el formato de la imagen y las dimensiones, haz el codigo mas friendly y que no sea tan exigente con el formato y dimensiones de las imágenes

- Aquí tienes la evaluación crítica redactada con un tono mucho más conversacional, analítico y natural, ideal para cerrar la memoria de la práctica:
- **Respuesta a la Pregunta 3.3 (Evaluación Crítica):**
- Haciendo balance entre los dos métodos, la verdad es que cada uno tiene su momento y lugar.
- **Ventajas de la red *Feed-Forward* (Bloque 1):** Su punto fuerte es evidente: es súper rápida, eficiente y facilísima de implementar en un entorno de producción real. Básicamente le pasas las imágenes y listo, te ahorras el dolor de cabeza de tener que ajustar hiperparámetros o esperar a que termine un bucle de entrenamiento pesadísimo.

- **Inconvenientes de la red *Feed-Forward*:** ¿El problema? Que te topas con una "caja negra". Te tienes que conformar con lo que te da porque pierdes toda la capacidad de personalización. No tienes control directo sobre lo agresivo que quieres que sea el estilo y tampoco puedes decidir qué capas de la red tienen más peso que otras.
- **Ventajas de la Optimización Explícita (Bloque 3):** Aquí es donde brilla el enfoque manual: nos da un control absoluto de la situación. Es un proceso transparente que nos permite auditar todo lo que ocurre, entender qué características están viendo las capas internas de la VGG19 y jugar con los pesos (los valores de  $\alpha$  y  $\beta$ ) para crear obras de arte súper personalizadas y ajustadas exactamente a lo que buscamos.
- **Inconvenientes de la Optimización Explícita:** La otra cara de la moneda es que es un método muy lento y que consume muchísimos recursos computacionales. Además, exige bastante callo técnico (saber aplicar bien el preprocesamiento, entender cómo fluyen los gradientes, elegir el *pooling* correcto...). Si no controlas estos detalles, es facilísimo que la red se atasque en un mínimo local y tu lienzo acabe siendo un borrón sin sentido, que es justo la frustración que vivimos inicialmente en el Bloque 2.